

# Problem Session 09

Dining Philosophers

# Instructions

- You are going to implement dining philosopher using semaphore
- You will have to implement from the design given in this slides otherwise the version you obtain from internet is not likely to contain deadlock
- The objective is to let deadlock occur, you need to capture and report the condition that leads to deadlock
- Then, solve deadlock problem and explain what part of codes that you add and how the code solves deadlock
- Form a group of up to 6 students
- Create report on **deadlock sequence** and **how to solve deadlock**
- Submit your report on LEB2

# Hint: The Dining Philosophers

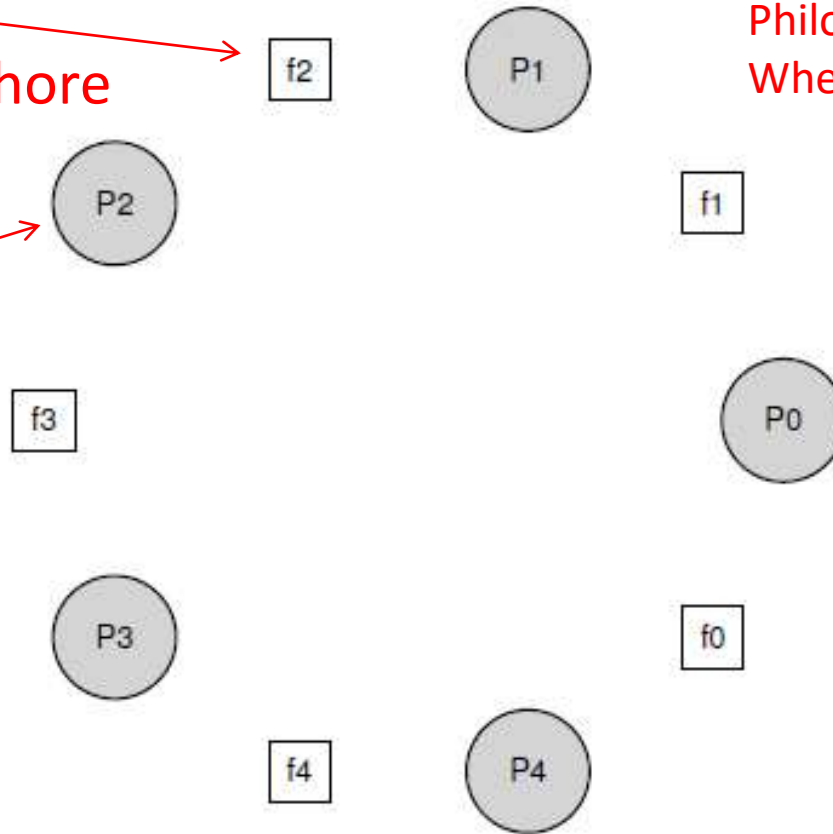
A fork  
is a semaphore

A philosopher  
is a thread

Philosophers either think or eat  
When eat, they need two forks

```
while (1)
{
    think();
    getforks(p);
    eat();
    putforks(p);
}
```

N philosophers with  
N forks



# Some Hints

- When philosopher  $p$  wishes to refer to the fork on their left, they simply call `left(p)`
- Similarly, the fork on the right of a philosopher is referred to by calling `right(p)`
  - `int left(int p) { return p; }`
  - `int right(int p) { return (p + 1) % 5; }`
- Declare the semaphores as follows:
  - `sem_t forks[5];`